user. Since we're testing auditing, we want to define at least a username that can be used as the principal.

Let's extend the `BookRepositoryJdbcTests` class with new test cases covering data auditing for users.

Listing 12.25 Testing data auditing when users are authenticated or not

```
@DataJdbcTest
@Import(DataConfig.class)
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@ActiveProfiles("integration")
class BookRepositoryJdbcTests {

  ...

  @Test                                                          ❶
  void whenCreateBookNotAuthenticatedThenNoAuditMetadata() {
    var bookToCreate = Book.of("1232343456", "Title",
      "Author", 12.90, "Polarsophia");
    var createdBook = bookRepository.save(bookToCreate);

    assertThat(createdBook.createdBy()).isNull();                ❷
    assertThat(createdBook.lastModifiedBy()).isNull();
  }

  @Test
  @WithMockUser("john")                                          ❸
  void whenCreateBookAuthenticatedThenAuditMetadata() {
    var bookToCreate = Book.of("1232343457", "Title",
      "Author", 12.90, "Polarsophia");
    var createdBook = bookRepository.save(bookToCreate);

    assertThat(createdBook.createdBy())
      .isEqualTo("john");                                        ❹
    assertThat(createdBook.lastModifiedBy())
      .isEqualTo("john");
  }
}
```

❶ This test case is executed in an unauthenticated context.

❷ No audit data when there is no authenticated user

❸ This test case is executed in an authenticated context for the user "john."

❹ Audit data when there is an authenticated user